

Teach Module 25: Service and Repository Layers

This README captures the Module 25 teaching session, practice exercises, and lab.

Module 25 focuses on organizing a Spring Boot application so that web/API code, business logic, and database access stay separated.

The common flow is:

```
Controller -> Service -> Repository -> Database
```

Each layer has a clear responsibility:

- **Controller** : handles HTTP requests and responses.
- **Service** : contains business rules and application workflows.
- **Repository** : talks to the database.
- **Entity** : represents database-backed data.
- **DTO** : represents data sent to or received from the API.

1. Why Layers Matter

Imagine an API endpoint that creates a bank account.

Poor design puts too much logic in the controller:

```
@RestController
public class AccountController {

    @PostMapping("/accounts")
    public Account create(@RequestBody Account account) {
        // validate account
        // check business rules
        // save to database
        // handle errors
        return accountRepository.save(account);
    }
}
```

This works, but the controller is doing too much. It knows about HTTP, business rules, and persistence.

Better design separates the work:

```
AccountController
    calls AccountService

AccountService
    applies business rules

AccountRepository
    handles database access
```

This makes the application easier to test, change, and debug.

2. Repository Layer

A repository is an abstraction over database operations.

With Spring Data JPA, you often write only an interface:

```
public interface AccountRepository extends JpaRepository<Account, Long> {  
    Optional<Account> findByAccountNumber(String accountNumber);  
    List<Account> findByCustomerId(Long customerId);  
}
```

Spring creates the implementation at runtime.

`JpaRepository<Account, Long>` means:

```
Account = entity type  
Long = primary key type
```

Common built-in methods include:

```
save(account)  
findById(id)  
findAll()  
deleteById(id)  
existsById(id)
```

Spring Data can also derive queries from method names:

```
findByStatus(String status)  
findByEmail(String email)  
findByLastNameAndActive(String lastName, boolean active)
```

3. Entity Example

An entity maps a Java class to a database table.

```
@Entity  
public class Account {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String accountNumber;  
  
    private BigDecimal balance;  
}
```

```
private String status;

// getters and setters
}
```

Important annotations:

- `@Entity` : this class is stored in the database.
- `@Id` : primary key.
- `@GeneratedValue` : database or JPA generates the ID.

4. Service Layer

The service layer is where business rules belong.

Example rule: a new account starts with a zero balance and active status.

```
@Service
public class AccountService {

    private final AccountRepository accountRepository;

    public AccountService(AccountRepository accountRepository) {
        this.accountRepository = accountRepository;
    }

    public Account createAccount(String accountNumber) {
        Account account = new Account();
        account.setAccountNumber(accountNumber);
        account.setBalance(BigDecimal.ZERO);
        account.setStatus("ACTIVE");

        return accountRepository.save(account);
    }
}
```

The service depends on the repository, but the controller should not directly manage database details.

5. Dependency Injection Across Layers

Spring injects dependencies for you.

This constructor:

```
public AccountService(AccountRepository accountRepository) {
    this.accountRepository = accountRepository;
}
```

means Spring provides an `AccountRepository` object when it creates `AccountService` .

Prefer constructor injection:

```
private final AccountRepository accountRepository;

public AccountService(AccountRepository accountRepository) {
    this.accountRepository = accountRepository;
}
```

Constructor injection is clear, testable, and avoids hidden dependencies.

Avoid field injection in modern Spring applications:

```
@Autowired
private AccountRepository accountRepository;
```

It works, but constructor injection is cleaner.

6. Controller Calling Service

```
@RestController
@RequestMapping("/accounts")
public class AccountController {

    private final AccountService accountService;

    public AccountController(AccountService accountService) {
        this.accountService = accountService;
    }

    @PostMapping
    public Account createAccount(@RequestBody CreateAccountRequest request) {
        return accountService.createAccount(request.accountNumber());
    }
}
```

Request DTO:

```
public record CreateAccountRequest(String accountNumber) {
}
```

The controller handles the web request. The service handles the application behavior.

7. Error Handling Across Layers

Suppose a user asks for an account that does not exist.

In the service:

```
public Account getAccount(Long id) {
    return accountRepository.findById(id)
```

```
        .orElseThrow(() -> new AccountNotFoundException(id));
    }
}
```

Custom exception:

```
public class AccountNotFoundException extends RuntimeException {
    public AccountNotFoundException(Long id) {
        super("Account not found: " + id);
    }
}
```

Translate that exception into an HTTP response:

```
@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(AccountNotFoundException.class)
    public ResponseEntity<String> handleAccountNotFound(AccountNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(ex.getMessage());
    }
}
```

Keep these boundaries clear:

- The repository should not decide HTTP status codes.
- The service should not build HTTP responses.
- The controller or advice layer handles API responses.

8. Persistence Boundary

The repository layer is the persistence boundary.

Database-specific work should stay near the repository layer.

This belongs in a repository:

```
Optional<Account> findByAccountNumber(String accountNumber);
```

This belongs in a service:

```
if (account.getBalance().compareTo(amount) < 0) {
    throw new InsufficientFundsException();
}
```

Simple rule:

- If it is about finding, saving, deleting, or querying data, use the repository.
- If it is about business meaning, validation, decisions, or workflow, use the service.

9. Common Mistakes

Putting business logic in controllers:

```
@PostMapping("/withdraw")
public Account withdraw(...) {
    // too much business logic here
}
```

Putting business logic in repositories:

```
public interface AccountRepository extends JpaRepository<Account, Long> {
    // repositories should not decide withdrawal rules
}
```

Returning entities directly from every API:

```
return account;
```

This can expose internal database structure. DTOs are often safer.

Making services pass-through wrappers:

```
public Account save(Account account) {
    return accountRepository.save(account);
}
```

A service should usually express a meaningful use case, not just duplicate repository methods.

Mental Model

Think of the service layer as the application brain.

Think of the repository layer as the database gateway.

- The controller asks: what does the user want?
- The service asks: what should happen?
- The repository asks: how do I store or retrieve the data?

Mini Practice

Design this feature:

```
Transfer money from one account to another.
```

You would likely need:

```
AccountRepository
    findById(...)
    save(...)

AccountService
```

```
transferMoney(fromId, toId, amount)
```

AccountController

POST /accounts/transfer

The service would handle rules:

- Amount must be positive.
- Source account must exist.
- Destination account must exist.
- Source account must have enough balance.
- Both accounts must be saved after the transfer.

Quick Check

1. Which layer should contain business rules?
2. Which layer talks to the database?
3. Why is constructor injection preferred?
4. Should a repository return `ResponseEntity` ?
5. Where should `AccountNotFoundException` be converted into a `404 Not Found` response?

Answers:

1. Service layer.
2. Repository layer.
3. It makes dependencies explicit and easier to test.
4. No. HTTP responses belong to controller/advice layers.
5. In a controller advice or exception handler.

Practice Exercises

Exercise 1: Build a Basic Repository

Create an `Employee` entity with:

```
id
firstName
lastName
email
department
salary
active
```

Create:

```
EmployeeRepository extends JpaRepository<Employee, Long>
```

Add query methods:

```
findByEmail(String email)
findByDepartment(String department)
```

```
findByActiveTrue()
findBySalaryGreaterThan(BigDecimal salary)
```

Goal: practice Spring Data repository method naming.

Exercise 2: Create a Service Layer

Create `EmployeeService` with methods:

```
createEmployee(...)
getEmployeeById(...)
getAllEmployees()
updateEmployee(...)
deactivateEmployee(...)
```

Rules:

- Employee email must be unique.
- Salary cannot be negative.
- Deactivate should set `active = false`, not delete the row.

Goal: keep business rules in the service layer.

Exercise 3: Add a Controller That Uses the Service

Create REST endpoints:

```
POST /employees
GET /employees
GET /employees/{id}
PUT /employees/{id}
PATCH /employees/{id}/deactivate
```

Rule: the controller must not call the repository directly.

Goal: practice the flow `Controller -> Service -> Repository`.

Exercise 4: Custom Exception Handling

Create exceptions:

```
EmployeeNotFoundException
DuplicateEmailException
InvalidSalaryException
```

Then create a `@RestControllerAdvice` class that maps them to:

```
EmployeeNotFoundException -> 404
DuplicateEmailException -> 409
InvalidSalaryException -> 400
```

Goal: separate business errors from HTTP response handling.

Exercise 5: DTO Mapping Practice

Do not expose the entity directly from the API.

Create:

```
CreateEmployeeRequest
UpdateEmployeeRequest
EmployeeResponse
```

Example:

```
public record CreateEmployeeRequest(
    String firstName,
    String lastName,
    String email,
    String department,
    BigDecimal salary
) {}
```

Goal: understand why API models and database models should often be separate.

Exercise 6: Repository Query Challenge

Add repository methods:

```
List<Employee> findByDepartmentAndActiveTrue(String department);

List<Employee> findByLastNameContainingIgnoreCase(String lastName);

boolean existsByEmail(String email);

Optional<Employee> findByEmailAndActiveTrue(String email);
```

Goal: practice readable query derivation.

Exercise 7: Service Workflow Challenge

Create a method:

```
transferEmployee(Long employeeId, String newDepartment)
```

Rules:

- Employee must exist.
- Employee must be active.
- New department cannot be blank.
- If the employee is already in that department, throw an exception or return unchanged.

Goal: practice workflow logic in the service layer.

Exercise 8: Unit Test the Service

Write unit tests for `EmployeeService` using Mockito.

Test cases:

```
createEmployee saves valid employee
createEmployee rejects duplicate email
createEmployee rejects negative salary
getEmployeeById throws when missing
deactivateEmployee changes active to false
```

Goal: prove that service logic can be tested without a real database.

Exercise 9: Repository Integration Test

Use `@DataJpaTest` to test repository queries.

Test:

```
findByEmail returns correct employee
findByDepartment returns only matching employees
existsByEmail works
findByActiveTrue excludes inactive employees
```

Goal: test repository behavior with an in-memory database.

Exercise 10: Refactor a Messy Controller

Start with a controller that directly uses `EmployeeRepository` .

Then refactor it into:

```
EmployeeController
EmployeeService
EmployeeRepository
```

Goal: understand why layering improves maintainability.

Lab: Service and Repository Layers

Build a small Employee Management API in Spring Boot.

Goal:

```
Controller -> Service -> Repository -> Database
```

The controller should never call the repository directly.

Lab Scenario

You are building an internal HR API that manages employees.

Each employee has:

```
id
firstName
lastName
email
department
salary
active
```

Step 1: Create the Entity

Create `Employee.java` .

```
@Entity
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String email;
    private String department;
    private BigDecimal salary;
    private boolean active = true;

    // getters and setters
}
```

Step 2: Create the Repository

Create `EmployeeRepository.java` .

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    Optional<Employee> findByEmail(String email);

    boolean existsByEmail(String email);

    List<Employee> findByDepartmentAndActiveTrue(String department);

    List<Employee> findByActiveTrue();
}
```

Step 3: Create Request and Response DTOs

Create `CreateEmployeeRequest.java` .

```
public record CreateEmployeeRequest(  
    String firstName,  
    String lastName,  
    String email,  
    String department,  
    BigDecimal salary  
) {}
```

Create `UpdateEmployeeRequest.java` .

```
public record UpdateEmployeeRequest(  
    String firstName,  
    String lastName,  
    String department,  
    BigDecimal salary  
) {}
```

Create `EmployeeResponse.java` .

```
public record EmployeeResponse(  
    Long id,  
    String firstName,  
    String lastName,  
    String email,  
    String department,  
    BigDecimal salary,  
    boolean active  
) {}
```

Step 4: Create Custom Exceptions

```
public class EmployeeNotFoundException extends RuntimeException {  
    public EmployeeNotFoundException(Long id) {  
        super("Employee not found: " + id);  
    }  
}
```

```
public class DuplicateEmailException extends RuntimeException {  
    public DuplicateEmailException(String email) {  
        super("Email already exists: " + email);  
    }  
}
```

```
public class InvalidEmployeeException extends RuntimeException {  
    public InvalidEmployeeException(String message) {  
        super(message);  
    }  
}
```

```
}  
}
```

Step 5: Create the Service

Create `EmployeeService.java` .

Required methods:

```
createEmployee(CreateEmployeeRequest request)  
getEmployeeById(Long id)  
getActiveEmployees()  
getEmployeesByDepartment(String department)  
updateEmployee(Long id, UpdateEmployeeRequest request)  
deactivateEmployee(Long id)
```

Business rules:

- Email must be unique.
- Salary cannot be negative.
- First name, last name, email, and department cannot be blank.
- Deactivate should not delete the employee.

Step 6: Create the Controller

Create endpoints:

```
POST    /employees  
GET     /employees  
GET     /employees/{id}  
GET     /employees/department/{department}  
PUT     /employees/{id}  
PATCH  /employees/{id}/deactivate
```

The controller should call only `EmployeeService` .

Step 7: Add Global Error Handling

Create `ApiExceptionHandler.java` .

Map exceptions:

```
EmployeeNotFoundException -> 404 Not Found  
DuplicateEmailException -> 409 Conflict  
InvalidEmployeeException -> 400 Bad Request
```

Step 8: Test with Sample Requests

Create employee:

```
POST /employees  
Content-Type: application/json
```

```
{
  "firstName": "Ava",
  "lastName": "Patel",
  "email": "ava.patel@example.com",
  "department": "Engineering",
  "salary": 95000
}
```

Update employee:

```
PUT /employees/1
Content-Type: application/json

{
  "firstName": "Ava",
  "lastName": "Patel",
  "department": "Platform Engineering",
  "salary": 105000
}
```

Deactivate employee:

```
PATCH /employees/1/deactivate
```

Success Criteria

You are done when:

- The app starts successfully.
- Employees can be created.
- Duplicate emails are rejected.
- Employees can be listed.
- Employees can be found by ID.
- Employees can be found by department.
- Employees can be updated.
- Employees can be deactivated without being deleted.
- Errors return proper HTTP status codes.
- The controller does not directly use `EmployeeRepository`.

Bonus Challenge

Add this endpoint:

```
PATCH /employees/{id}/transfer
```

Request body:

```
{
  "department": "Cloud Engineering"
}
```

```
}
```

Rules:

- Employee must exist.
- Employee must be active.
- Department cannot be blank.
- Employee cannot be transferred to the same department.